PROJECT REPORT

CLOTHING SWAP PLATFORM

Submitted By

Vikyath V Kotian

Internship

Unified Mentor Pvt .Ltd.

Internship Duration

June 15 (2026) - August 15 (2026)

Submitted On

August 15 (2026)

Table of Contents

1. Project Overview.....1
2. Objectives.....2
3. Technology Stack.....3
4. System Architecture.....4
5. Features.....5
6. Screenshots.....8
7. Deployment.....13
8. Challenges.....14
9. Future Enhancements.....15
10. Conclusion.....16

Project Overview

The **Clothing Swap Platform** is a full-stack web application developed to encourage sustainable fashion by enabling users to exchange clothing items instead of purchasing new ones. The platform provides a convenient and secure environment where users can create accounts, upload clothing listings, browse items shared by others, and request clothing swaps based on their preferences. By promoting the reuse of wearable clothing, the application aims to reduce textile waste while making fashion more accessible and environmentally responsible.

The application has been designed with separate functionalities for users and administrators to ensure efficient platform management. Registered users can manage their personal profiles, upload and maintain clothing listings, initiate swap requests, communicate with other users through an integrated chat system, receive notifications regarding platform activities, and track the progress of their ongoing and completed swaps. On the administrative side, dedicated tools are available to monitor user activities, manage listings, resolve disputes, and view platform analytics, ensuring that the platform remains secure, organized, and user-friendly.

The project was developed using a modern web development technology stack. The frontend was built with **React.js** to provide a responsive and interactive user interface, while the backend was implemented using **Node.js** and **Express.js** for handling business logic and RESTful APIs. **MongoDB Atlas** was used as the cloud database for storing user information, listings, deals, chats, notifications, and dispute records. Authentication and authorization were implemented using **JSON Web Tokens (JWT)** to ensure secure access to protected resources. Finally, the application was deployed using **Vercel** for the frontend and **Render** for the backend, making the platform accessible over the internet.

Throughout the development process, emphasis was placed on building a practical, scalable, and user-centric application by following industry-standard development practices. The project involved implementing secure authentication, database integration, cloud deployment, REST API development, and responsive user interface design while addressing real-world challenges such as deployment configuration, route handling, API integration, and access control. The successful completion of this project provided valuable hands-on experience in full-stack web development and demonstrated the ability to design, develop, deploy, and maintain a complete web application from start to finish.

Objectives

- To develop a secure and user-friendly web application for clothing exchange.

- To enable users to register, log in, and securely manage their profiles.

- To provide functionality for uploading, editing, and managing clothing listings.

- To facilitate clothing swap requests between users through a simple and efficient workflow.

- To implement real-time chat and notification features for effective communication.

- To develop an administrative dashboard for managing users, listings, disputes, and platform analytics.

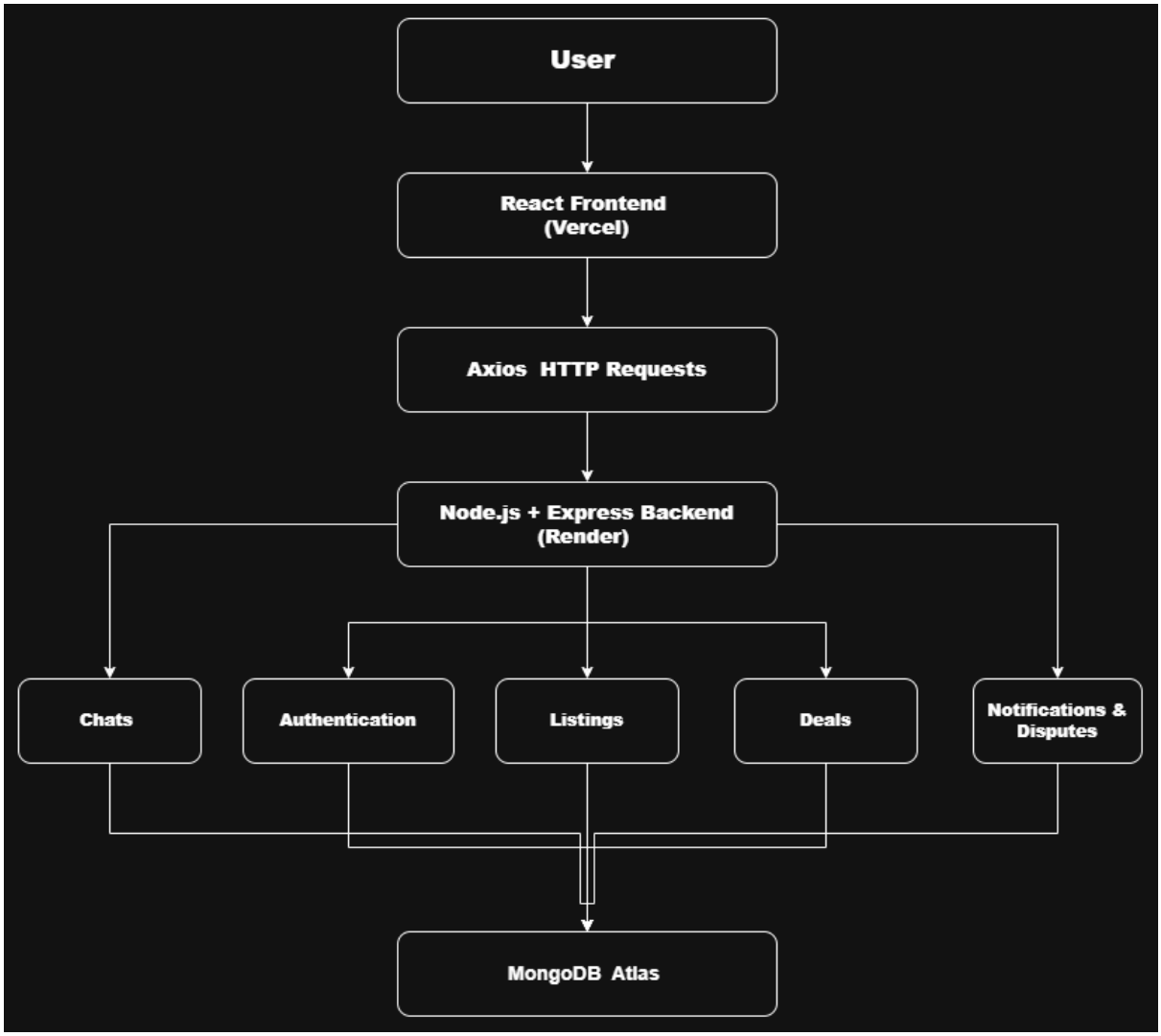
- To ensure secure authentication and authorization using JSON Web Tokens (JWT).

- To deploy the application on cloud platforms using Vercel and Render for public accessibility and scalability.

Technology Stack

Technology	Purpose
React	Frontend Development
Node.js	Backend Runtime Environment
Express.js	RESTful API Development
MongoDB Atlas	Cloud Database
JWT	User Authentication & Authorization
Axios	API Communication
Vercel	Frontend Deployment
Render	Backend Deployment

System Architecture



The Clothing Swap Platform follows a client-server architecture where users interact with the application through a React-based frontend hosted on Vercel. User actions, such as registration, login, uploading listings, and requesting swaps, are sent to the backend via Axios HTTP requests. The backend, developed using Node.js and Express.js and deployed on Render, processes these requests by executing the required business logic. It handles various functional modules, including authentication, clothing listings, swap deals, chat, notifications, and dispute management. The backend communicates with MongoDB Atlas using Mongoose to store and retrieve application data securely. This layered architecture ensures a clear separation between the presentation layer, application logic, and data storage, making the system scalable, maintainable, and easy to extend with additional features in the future.

Features Implemented

1. User Authentication

The Clothing Swap Platform provides a secure user authentication system that allows users to create an account and log in using their registered email address and password. Passwords are securely stored after encryption, ensuring that sensitive user information remains protected. After successful authentication, a JSON Web Token (JWT) is generated to verify the user's identity and authorize access to protected features of the application. This mechanism ensures that only authenticated users can access personalized services such as managing listings, requesting swaps, chatting with other users, and viewing notifications.

2. Clothing Listings Management

The platform enables users to upload clothing items that they wish to exchange with others. Each listing contains relevant details such as the clothing title, description, category, condition, size, and image. Users can view all available listings, edit their own listings whenever required, or remove listings that are no longer available for swapping. This feature allows users to effectively manage their wardrobe while making the platform organized and easy to browse.

3. Swap Request Management

Users can browse clothing items uploaded by other members and initiate swap requests for items they are interested in. The recipient of the request has the option to accept or reject the proposed swap. Once a request is accepted, the deal is recorded and becomes visible in the "My Deals" section, allowing both users to monitor the progress of their clothing exchange. This feature forms the core functionality of the platform by enabling secure and organized clothing swaps between users.

4. Real-Time Chat

To facilitate effective communication between users, the platform includes an integrated chat system. Once a swap request is accepted, users can exchange messages directly through the application to discuss details related to the clothing exchange. The chat functionality improves coordination between users while eliminating the need for external communication platforms, thereby providing a seamless user experience.

5. Notification System

The application provides a notification system that informs users about important activities occurring on the platform. Notifications are generated for events such as incoming swap requests, accepted deals, rejected requests, and other significant updates. This feature helps users stay informed about the status of their interactions without needing to repeatedly check different sections of the application.

6. User Profile Management

Every registered user has access to a personal profile page where important account information and platform activity are displayed. The profile includes user details, pending swap requests, completed swap history, and quick navigation to important sections such as My Deals. This centralized profile management system provides users with a convenient overview of their activity on the platform.

7. Administrative Dashboard

The platform includes a dedicated administrative dashboard that enables efficient monitoring and management of the entire application. Administrators

have access to user management, clothing listing management, dispute resolution, platform analytics, and monitoring of swap activities. These administrative tools help maintain platform integrity, ensure compliance with community guidelines, and provide insights into overall platform usage.

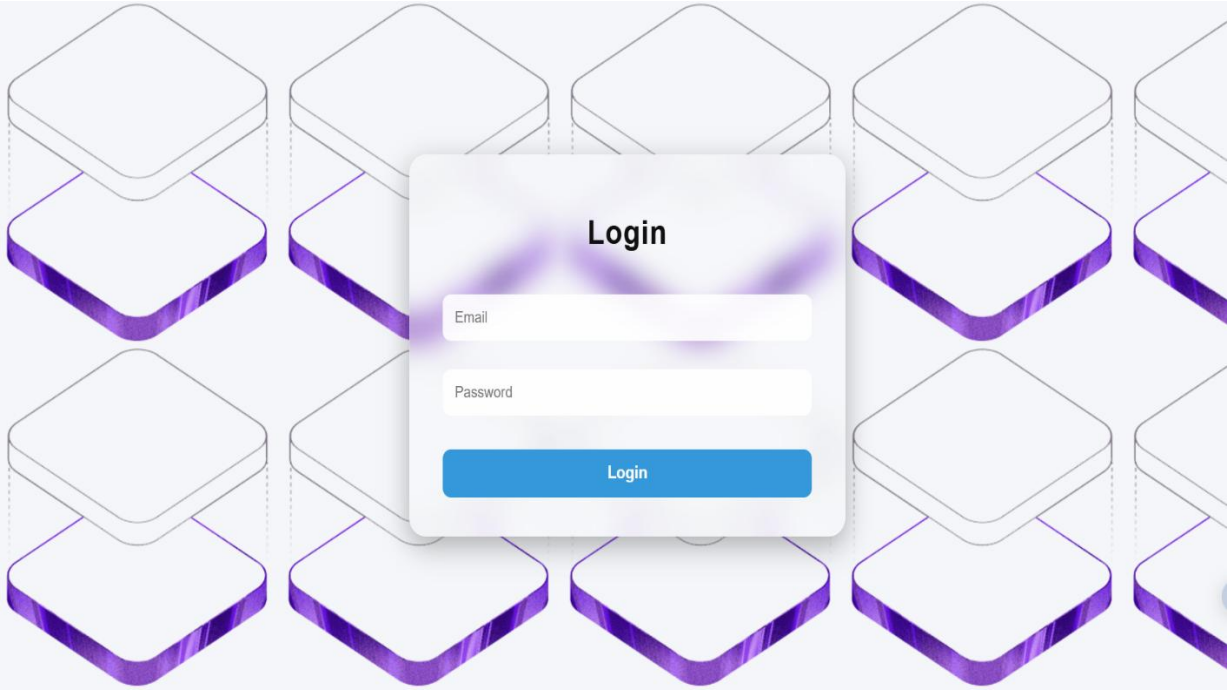
8. Dispute Management

To ensure fairness and user trust, the application includes a dispute management module. Users can raise disputes regarding completed or ongoing swap transactions by providing an explanation of the issue encountered. Administrators can review submitted disputes and update their status as resolved or rejected. This feature improves transparency and strengthens user confidence in the platform.

9. Cloud Deployment

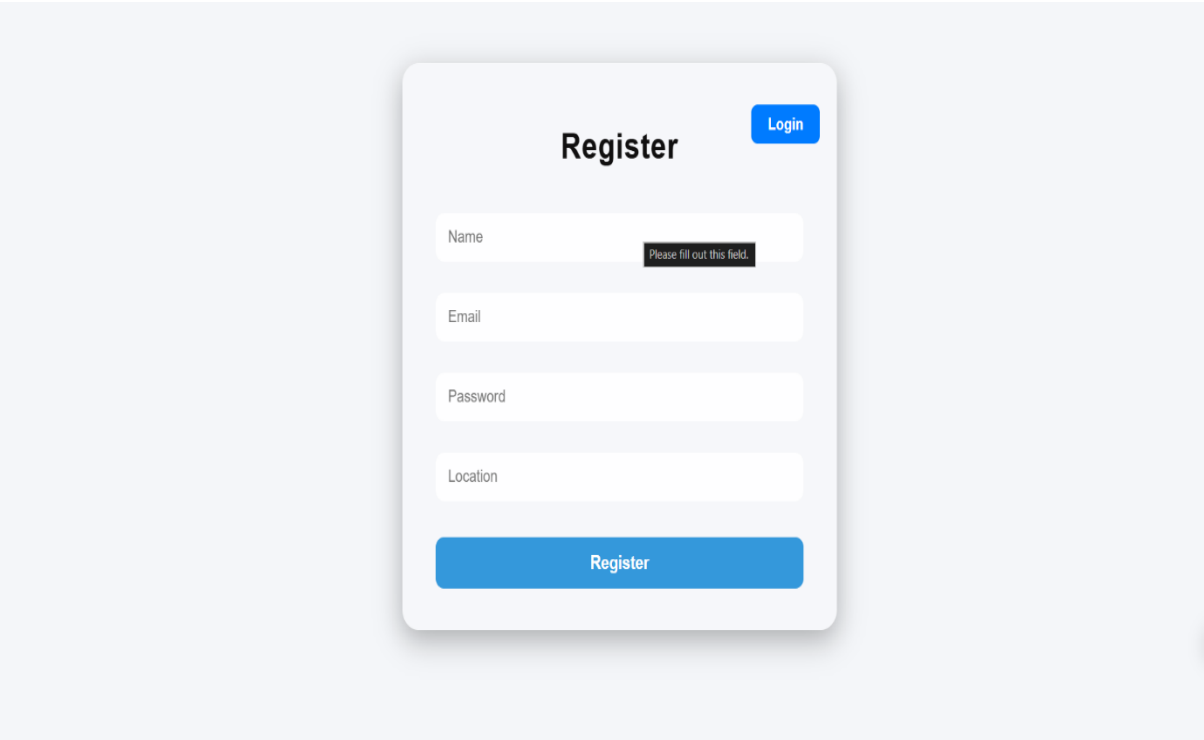
The Clothing Swap Platform has been successfully deployed using modern cloud hosting services. The frontend application is hosted on Vercel, while the backend API is deployed on Render. MongoDB Atlas is used as the cloud database for securely storing all application data. This deployment architecture allows users to access the platform online from any location while ensuring scalability, reliability, and efficient application performance.

Login Page



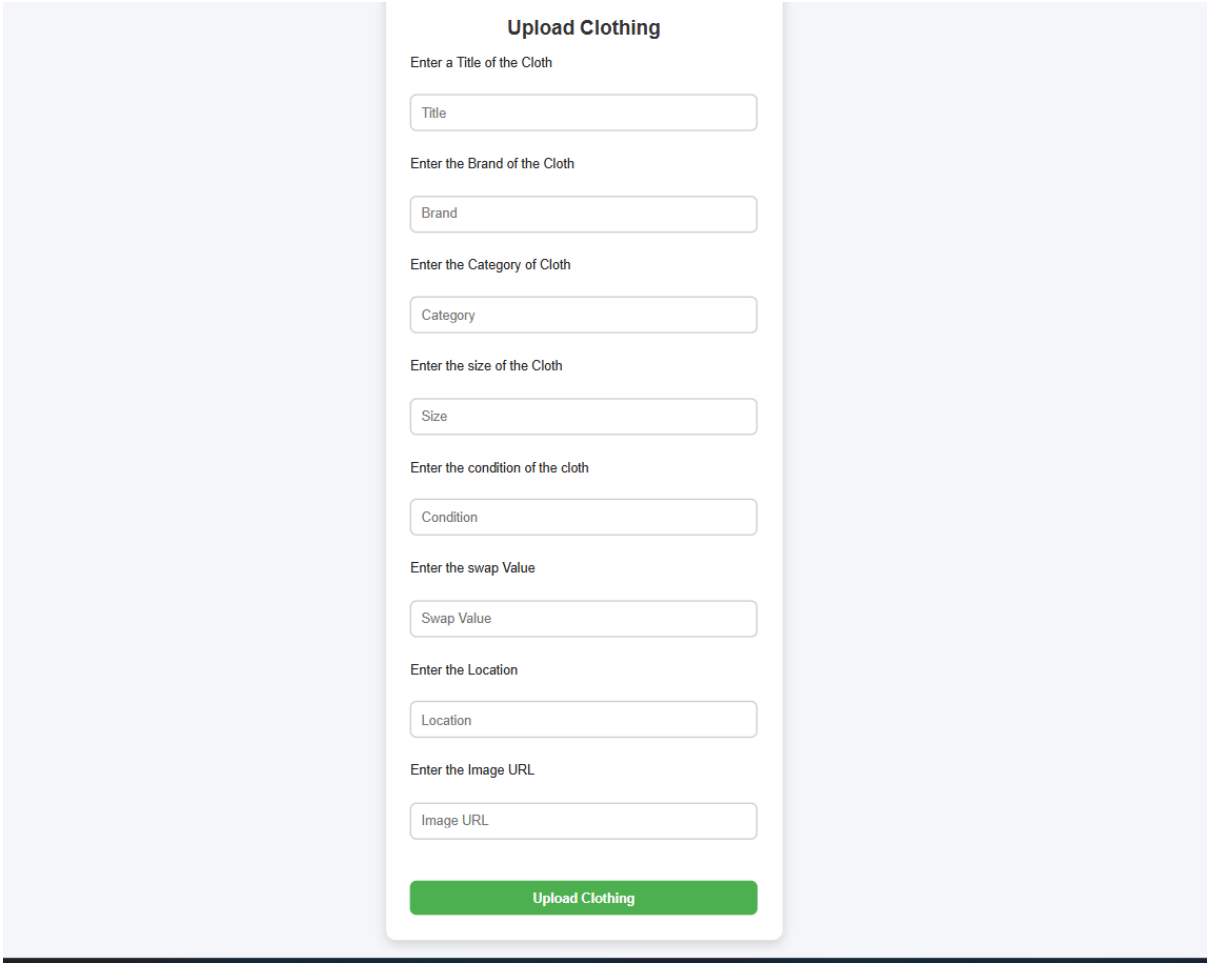
The Login page provides a secure authentication mechanism for registered users. Users enter their registered email address and password to access their account. Upon successful authentication, a JSON Web Token (JWT) is generated to verify the user's identity and authorize access to protected sections of the application. Invalid credentials are handled gracefully by displaying appropriate error messages, ensuring both security and a user-friendly experience.

Register Page



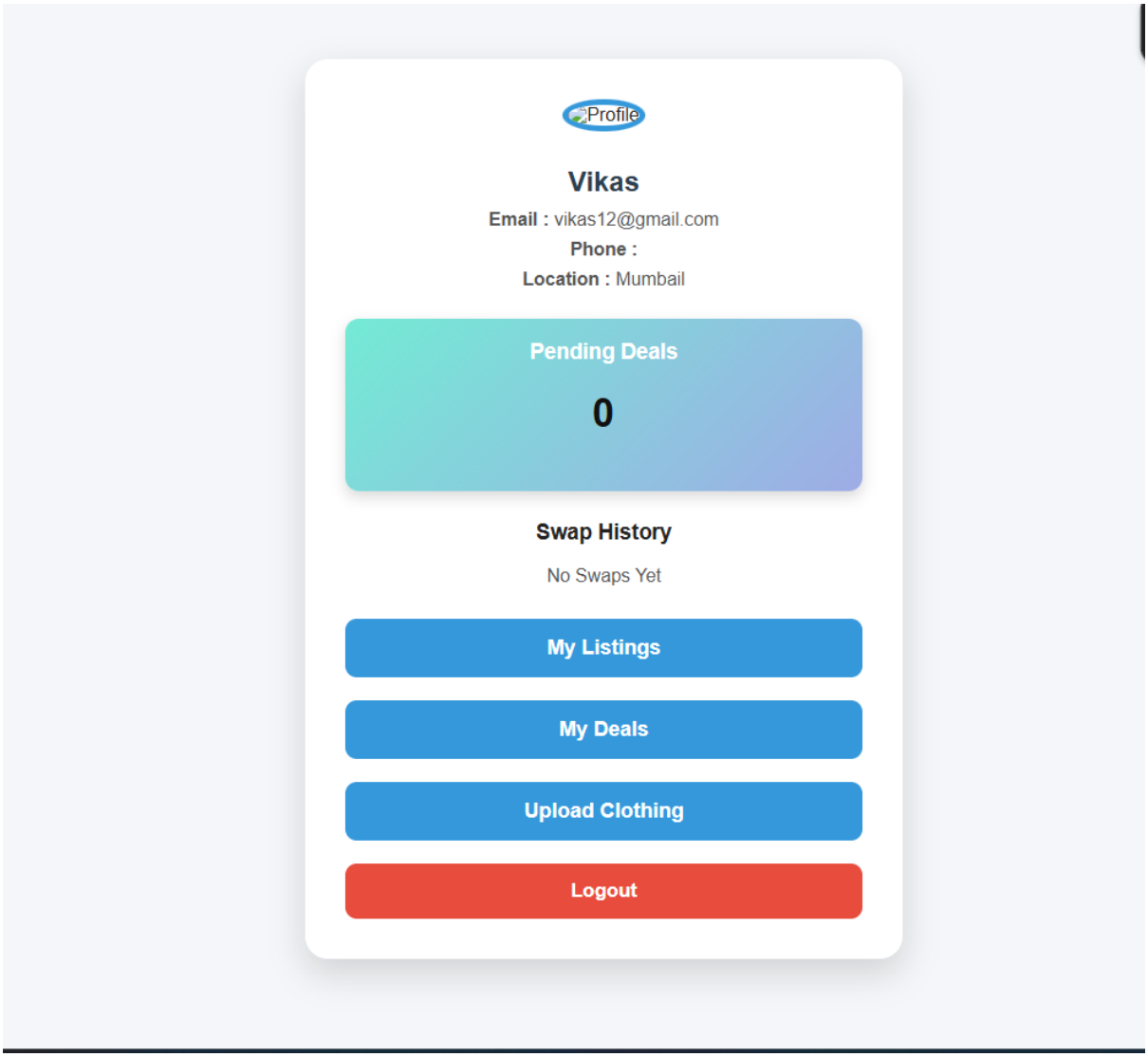
The Registration page enables new users to create an account by providing their personal information such as name, email address, and password. The application validates user input to ensure that required fields are completed correctly before storing the information securely in the database. This process allows only valid users to join the Clothing Swap Platform.

Upload Listing



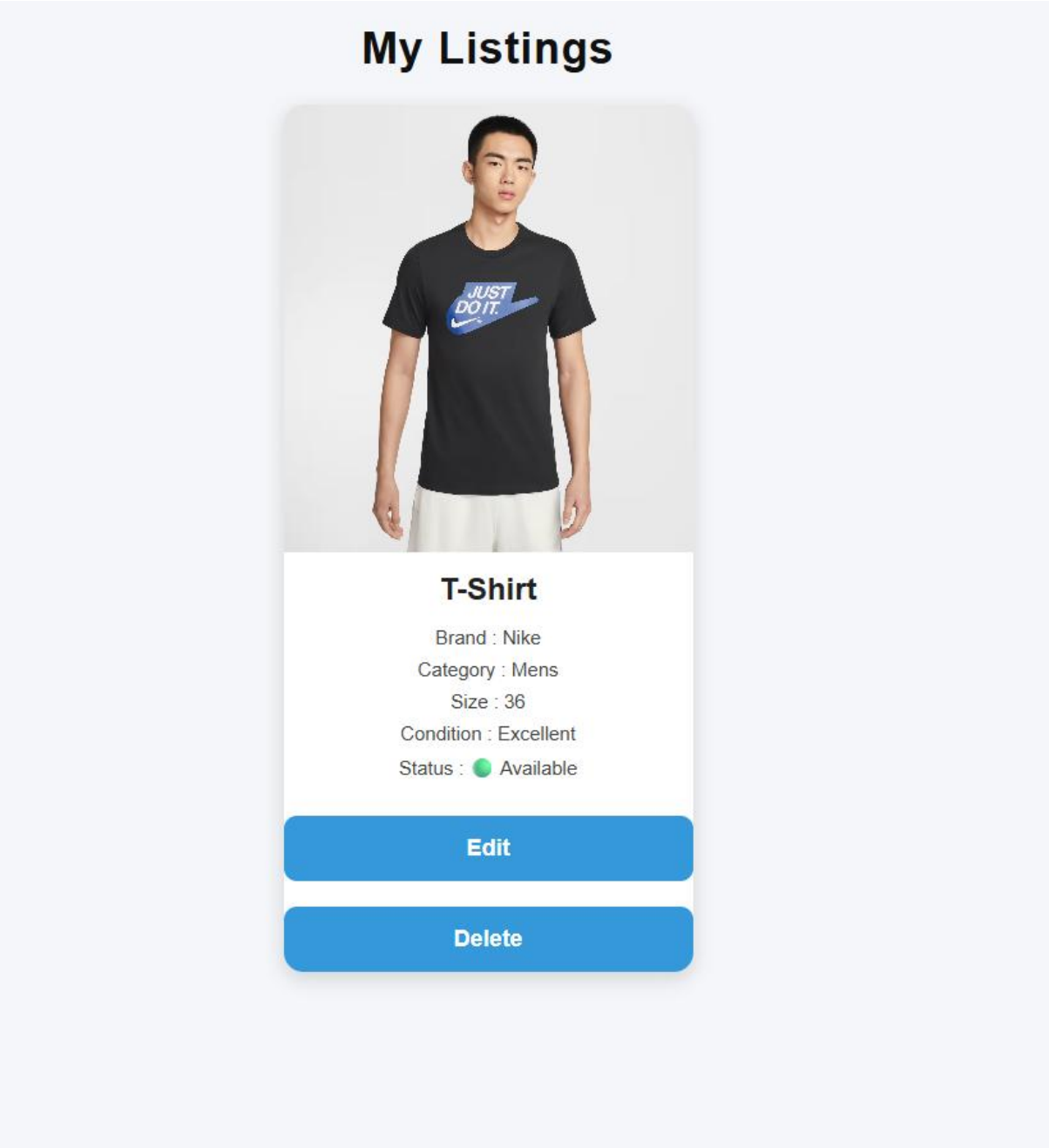
The Upload Listing page allows authenticated users to add clothing items available for exchange. Users provide information including the clothing title, category, condition, description, and image. Once submitted, the listing is stored in the database and becomes visible to other users for potential swap requests.

MyProfile



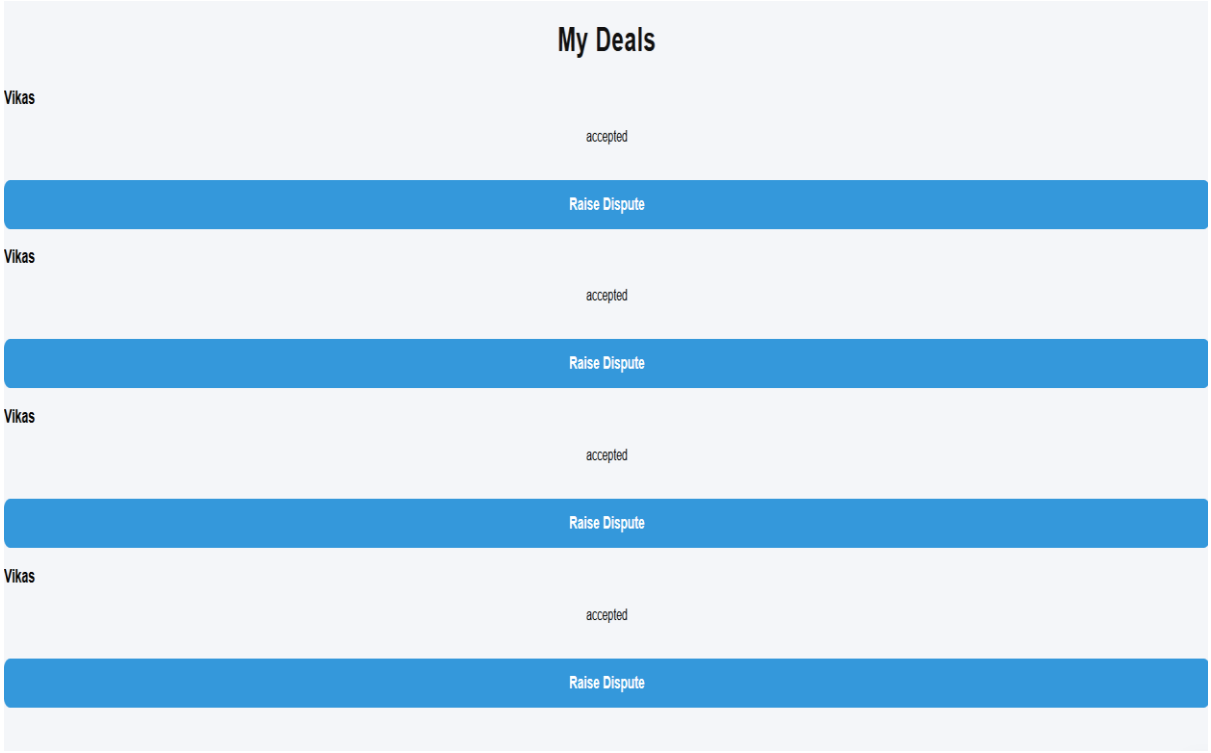
The **Profile Page** serves as a personalized dashboard where users can view and manage their account information in a centralized location. It displays essential user details, including the user's name and email address, along with a summary of platform activities such as pending swap requests and completed swap history. The page also provides quick navigation to the **My Deals** section, enabling users to efficiently monitor their ongoing and completed clothing exchanges. By consolidating account information and user activity into a single interface, the Profile Page enhances usability and provides users with a clear overview of their participation on the Clothing Swap Platform.

My Listings



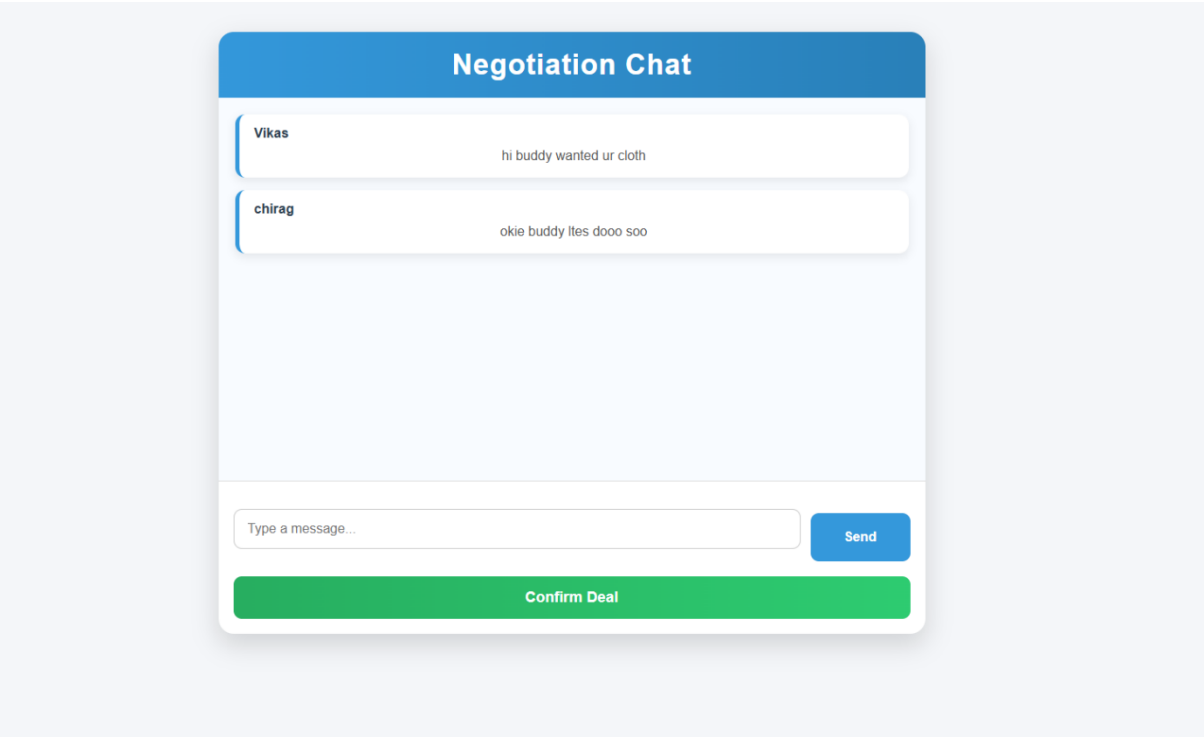
The My Listings page displays all clothing items uploaded by the logged-in user. It provides options to edit or delete existing listings, enabling users to keep their wardrobe listings accurate and up to date. This feature offers complete control over the items available for swapping.

My Deals



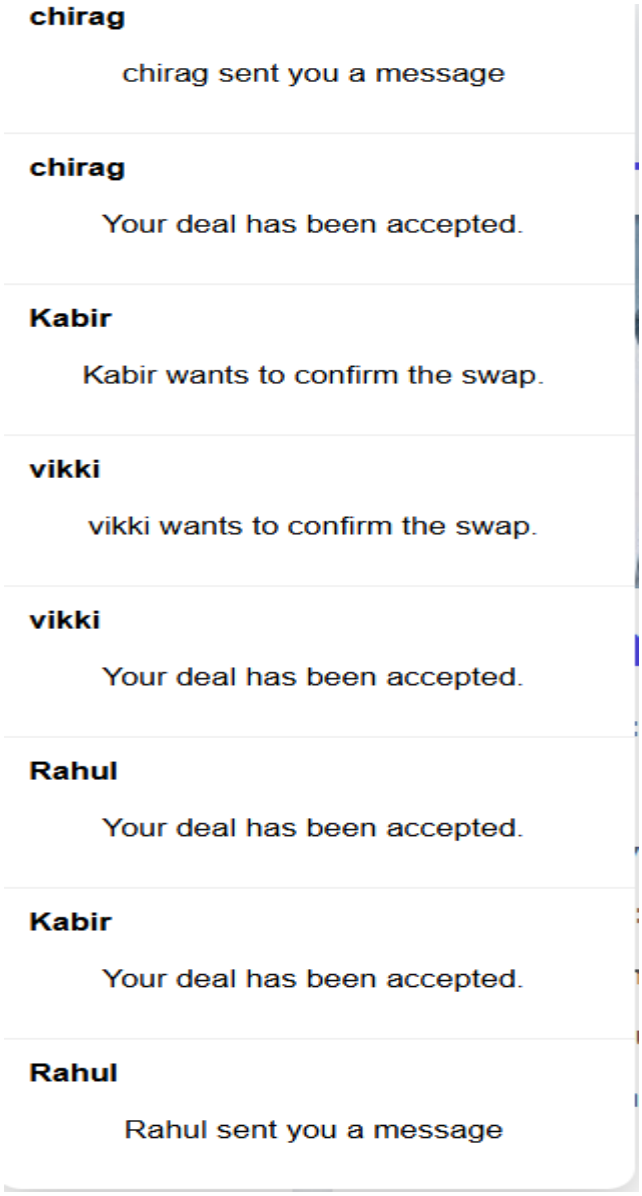
The My Deals page presents all accepted clothing swap requests involving the logged-in user. It allows users to monitor the status of their ongoing and completed exchanges, providing a centralized location for managing swap activities and maintaining a history of successful deals.

Chat



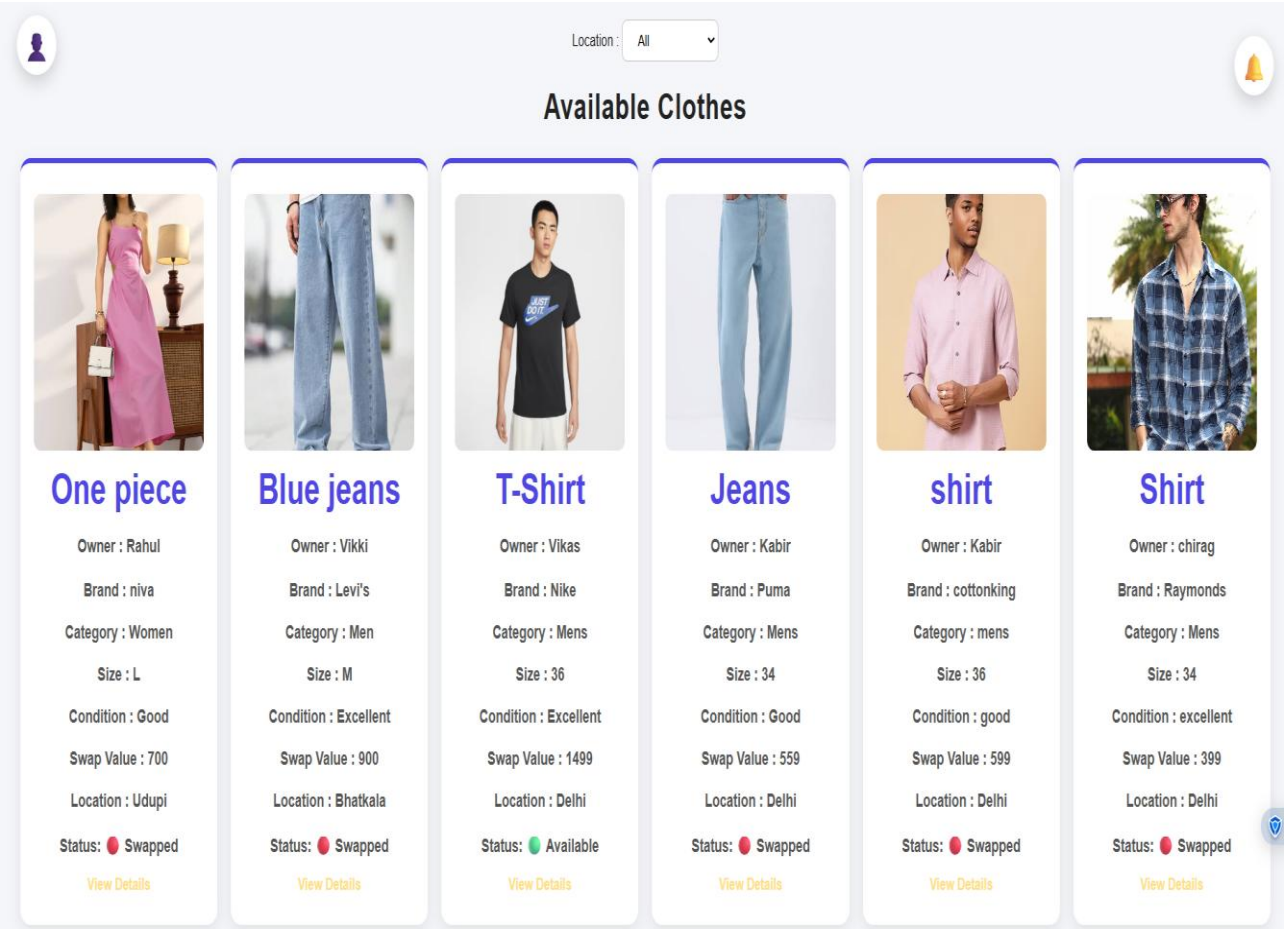
The integrated chat module enables users involved in an accepted swap request to communicate directly within the platform. This feature facilitates discussions regarding exchange details, meeting arrangements, or any additional information required to complete the clothing swap efficiently.

Notifications



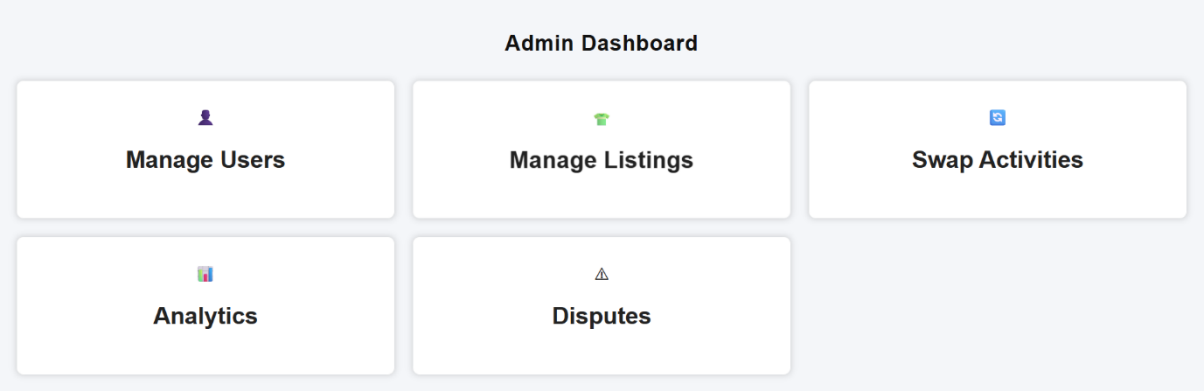
The Notification page informs users about important platform activities, including new swap requests, accepted or rejected deals, and other relevant updates. The notification system helps users remain informed without manually checking every section of the application.

Home



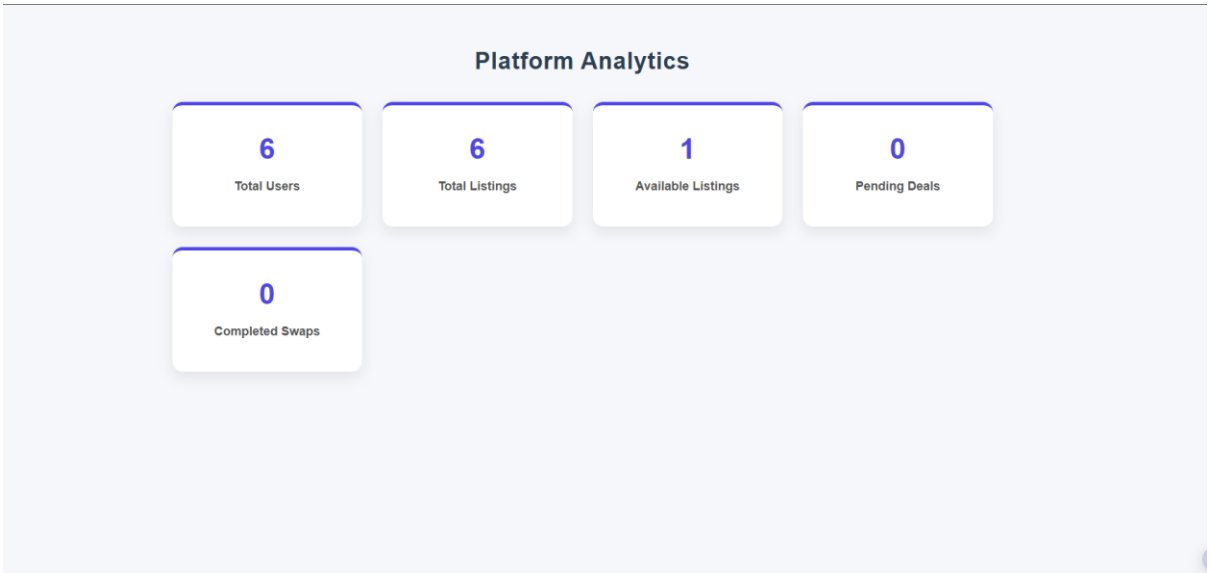
The **Home Page** serves as the primary interface of the Clothing Swap Platform, providing users with access to all available clothing listings uploaded by members of the platform. It offers a clean and user-friendly layout that allows users to browse clothing items along with their details, such as category, condition, size, and description. The page also includes navigation options that enable users to quickly access different modules of the application, including Profile, My Listings, Notifications, and other essential features. By presenting all available listings in a centralized view, the Home Page helps users easily discover clothing items of interest and initiate swap requests, thereby providing a smooth and engaging user experience.

Admin Dashboard



The Admin Dashboard serves as the central control panel for platform administration. It provides quick access to user management, listing management, dispute handling, swap activity monitoring, and platform analytics. This centralized interface enables administrators to efficiently oversee platform operations

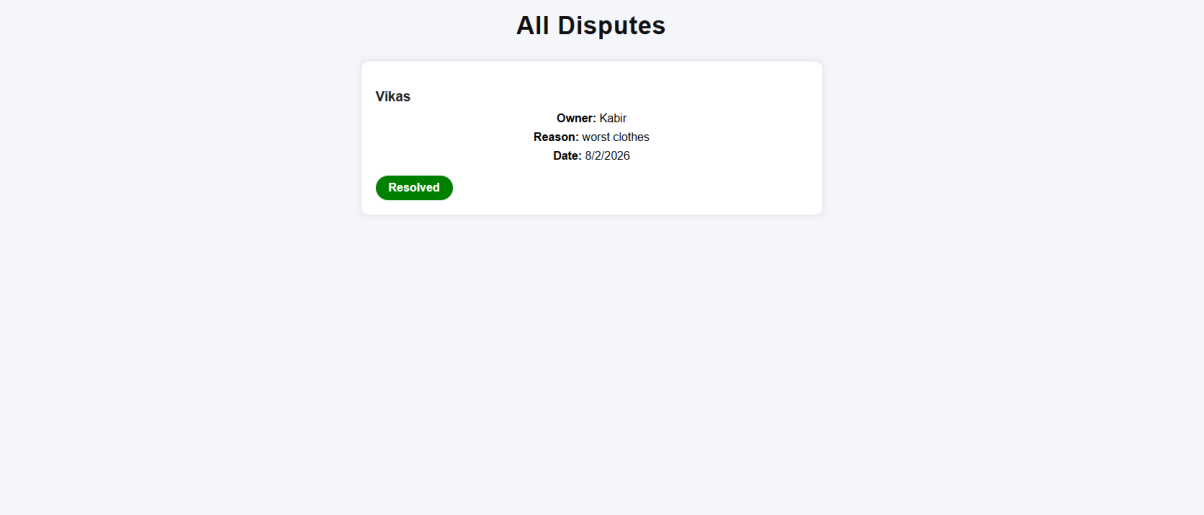
Platform Analytics



The Platform Analytics section presents statistical information about the application's overall usage. Administrators can monitor the number of registered users, clothing listings, completed swap transactions, and other key performance

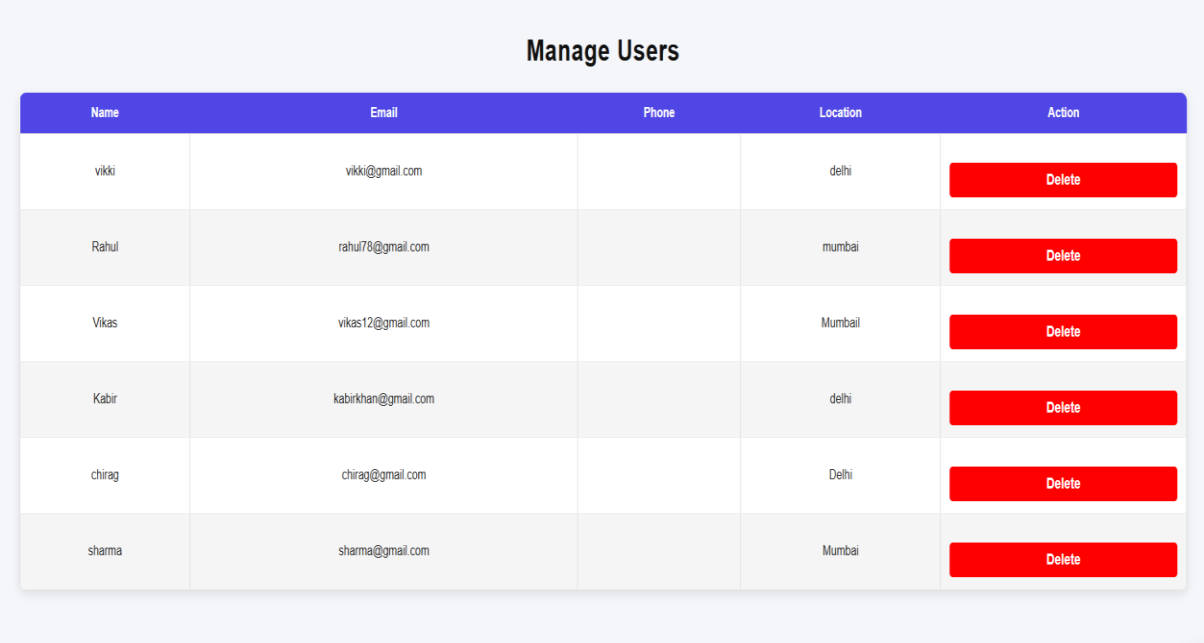
indicators. These insights assist in evaluating platform growth and user engagement.

Dispute Management



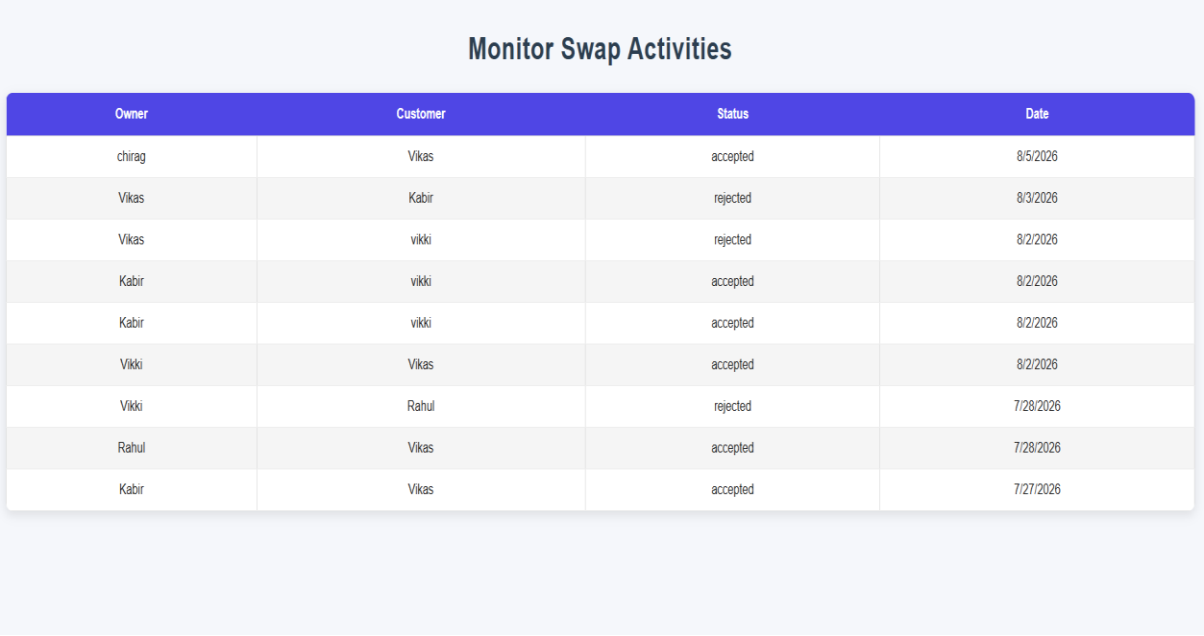
The Dispute Management module enables administrators to review disputes submitted by users regarding clothing swap transactions. Administrators can examine the reported issue and update its status as resolved or rejected, ensuring transparency and fairness across the platform.

Manage Users



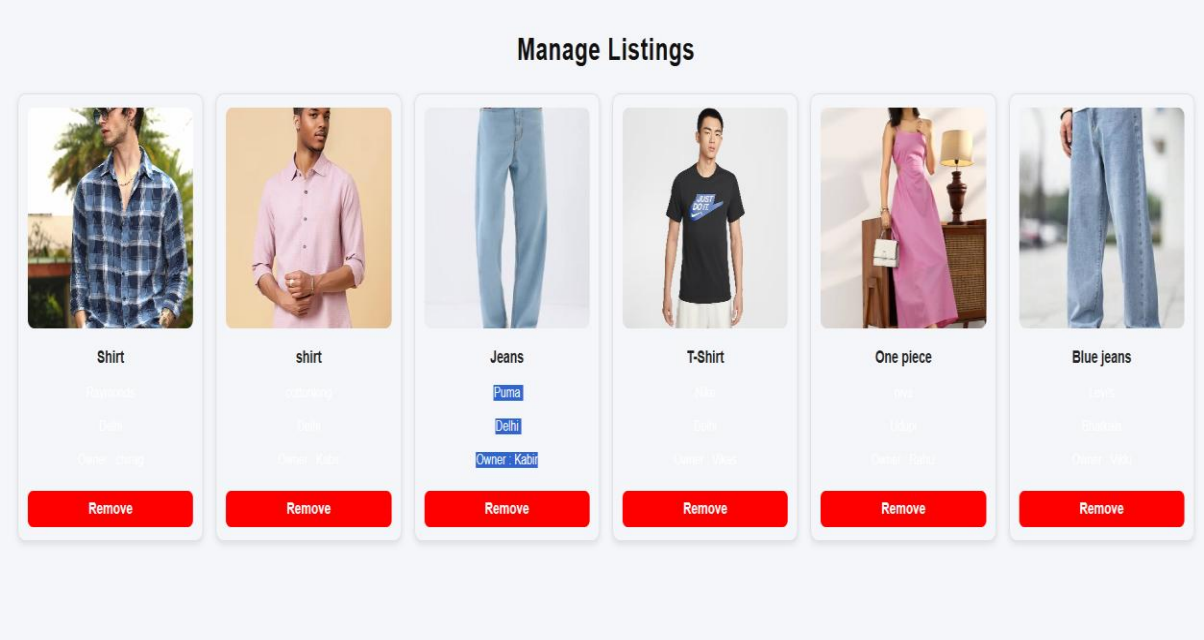
The **Manage Users** page provides administrators with a centralized interface to monitor and manage all registered users on the Clothing Swap Platform. It displays important user information such as the user's name, email address, and account details, allowing administrators to efficiently oversee platform membership. The module also provides the ability to remove user accounts that violate platform guidelines or are no longer required, helping maintain a secure and well-managed community. By offering complete visibility into registered users, this feature enables administrators to ensure smooth platform operations while promoting a safe and reliable environment for all users.

Monitor Swap Activities



The **Monitor Swap Activities** page enables administrators to track all clothing swap transactions occurring on the platform. It provides a comprehensive overview of swap requests, including the users involved, the clothing items being exchanged, and the current status of each swap, such as pending, accepted, or completed. This module allows administrators to monitor the overall workflow of clothing exchanges, identify unusual activities, and ensure that transactions are progressing as expected. By maintaining visibility over all swap operations, the feature contributes to platform transparency, operational efficiency, and effective administrative oversight.

Manage Listings



The **Manage Listings** page allows administrators to view and manage all clothing listings submitted by users across the platform. It displays detailed information about each listing, including the clothing title, category, condition, and the user who uploaded it. Administrators have the authority to remove inappropriate, duplicate, or inactive listings that do not comply with platform policies, ensuring that the marketplace remains organized and trustworthy. This feature helps maintain the quality of available listings while providing users with a better browsing experience.

Deployment

Deployment is the final stage of the software development lifecycle, where the application is made accessible to end users over the internet. The Clothing Swap Platform has been successfully deployed using cloud-based hosting services to ensure scalability, reliability, and continuous availability. The frontend, backend, and database components have been hosted on different cloud platforms, enabling seamless communication between all layers of the application and providing users with a smooth online experience.

Component	Platform	Status
Frontend	Vercel	Successfully Deployed
Backend	Render	Successfully Deployed
Database	MongoDB Atlas	Connected and Operational

Deployment Links

Frontend URL : clothing-swap-platform-chi.vercel.app

Backend URL : **https://clothing-swap-platform.onrender.com**

GitHub Repository Link : https://github.com/Vikyath301/Clothing-Swap-Platform.git

Challenges Faced and Solutions

During the development and deployment of the Clothing Swap Platform, several technical challenges were encountered across the frontend, backend, database and authentication. These challenges provided practical experience in identifying errors, debugging application behavior, configuring development environments, and deploying a full-stack application to the cloud. Each issue was analyzed systematically and resolved by modifying the relevant application configuration or implementation. The major challenges encountered during the project and the approaches used to resolve them are described below.

Sr No.	Challenge	Solution
1	CORS Configuration Issues	Configured the backend CORS policy to allow requests from the deployed frontend and local development environment.
2	JWT Authentication and Cookies	Verified JWT generation, cookie configuration, middleware validation, and environment-based secret configuration.
3	Vercel Routing / 404 Error	Configured frontend routing so that direct navigation and page refreshes correctly load the React application.
4	Backend Deployment on Render	Configured the Node.js/Express application to use the hosting platform's assigned port and production environment settings.

1 . CORS Configuration Issues

Challenge

One of the major challenges encountered during deployment was the Cross-Origin Resource Sharing (CORS) configuration. During local development, the frontend and backend were running on different localhost ports, and after deployment they were hosted on different domains. As a result, browser security policies prevented certain requests from the frontend from reaching the backend. This resulted in CORS-related errors when performing operations such as user registration, login, and retrieving application data.

Solution

The issue was resolved by configuring the CORS middleware in the Express.js backend to allow requests from the appropriate frontend origins. The configuration was updated to support both the local development environment and the deployed frontend URL. After making the necessary changes, API requests from the React frontend were successfully processed by the backend.

2 . JWT Authentication and Cookies

Challenge

Implementing authentication across the deployed frontend and backend was another important challenge. The application uses JSON Web Tokens (JWT) to authenticate users and protect restricted routes. During testing, authentication-related errors occurred when the backend was unable to retrieve or validate the token stored in the browser cookie. This resulted in unauthorized responses when accessing protected resources such as the user profile and administrative features.

Solution

The issue was investigated by checking the JWT generation process, cookie configuration, authentication middleware, and token verification logic. The JWT secret was moved to an environment variable instead of being hard-coded, and the authentication middleware was configured to correctly read and verify the token. The frontend and backend authentication flow was then tested again to ensure that authenticated users could access their respective protected resources.

3 . Vercel Routing and 404 Error

Challenge

fter deploying the React application, the website worked correctly when navigating through the application, but refreshing certain routes resulted in a 404: NOT_FOUND error. This occurred because the frontend application uses client-side routing through React Router. When a user navigates to a route such as /profile, React handles the route in the browser. However, when the page is refreshed, the hosting server attempts to locate /profile as an independent resource instead of redirecting the request to the React application's entry point.

Solution

The issue was resolved by configuring the deployment environment to redirect unknown frontend routes to the application's main entry point. This allowed React Router to handle the requested route on the client side. After applying the routing configuration, users were able to refresh application pages and directly access frontend routes without receiving the 404 error.

4 . Render Backend Deployment

Challenge

Deploying the Node.js and Express backend to Render also required changes to the production configuration. The backend had been developed and tested locally using a local server configuration, while the cloud hosting environment provides its own runtime and port configuration. Incorrect server configuration can prevent the backend from starting or make it inaccessible to the frontend**.**

Solution

The backend was configured to listen on the port provided by the hosting environment, while required environment variables were configured through the Render dashboard. The deployment logs were monitored to identify startup and configuration issues. After correcting the configuration, the backend successfully started and became accessible through its deployed API URL.

Key Learning from the Challenges

The challenges encountered during the project provided practical experience in full-stack application debugging and deployment. They helped develop a better understanding of how frontend applications communicate with backend APIs, how authentication is maintained across different environments, and how cloud hosting platforms handle application routing and configuration. The deployment process also provided hands-on experience with environment variables, database connectivity, CORS policies, API debugging, and production configuration. Resolving these issues improved both the technical implementation of the project and the ability to systematically diagnose and solve real-world software development problems.

Future Enhancements

Although the Clothing Swap Platform has been successfully developed and deployed with the major required functionalities, there are several opportunities to further improve the platform in the future. These enhancements can focus on improving user convenience, personalization, communication, security, and overall platform scalability. The following features can be considered in future versions of the application to provide a more comprehensive and engaging clothing exchange experience.

1 . Mobile Application

A dedicated mobile application can be developed for Android and iOS platforms to provide users with easier access to the clothing exchange service. A mobile application would allow users to upload clothing items, browse listings, receive notifications, communicate with other users, and manage swap requests directly from their smartphones. Mobile-specific features such as camera integration could also make it easier for users to capture and upload clothing images.

2 . AI – Based Recommendations

An AI-based recommendation system could be introduced to provide personalized clothing suggestions based on users' preferences, previous interactions, browsing behavior, categories of interest, and swap history. The system could analyze available listings and recommend items that are more relevant to each user. This would reduce the time required to search through listings and could improve user engagement with the platform.

3 . Payment Integration

Although the current platform focuses primarily on clothing exchanges, future versions could include secure payment functionality for users who want to purchase items or pay additional charges associated with certain transactions. Integration with trusted payment gateways could support digital payment methods while incorporating appropriate security measures for protecting transaction information.

4 . Rating and Review System

A rating and review system could be introduced to allow users to provide feedback after completing a clothing swap. Users could rate their experience and provide comments regarding the reliability, communication, and overall transaction experience of another user. A transparent rating system could help establish trust within the community and encourage responsible participation on the platform.

5 . Push Notifications

The existing notification functionality can be further enhanced through push notifications. Users could receive instant alerts on their mobile devices when they receive a swap request, when a request is accepted or rejected, when a new message is received, or when there is an update regarding a dispute. This would help users remain informed even when they are not actively using the application.

Conclusion

The Clothing Swap Platform was successfully designed, developed, and deployed as a full-stack web application during the internship. The project provides users with a centralized platform for exchanging clothing items, allowing them to register and authenticate securely, create and manage clothing listings, browse items, initiate swap requests, communicate with other users, receive notifications, and monitor their deals. In addition to the user-facing functionalities, an administrative module was implemented to support user management, listing management, swap activity monitoring, platform analytics, and dispute resolution. The successful integration of these modules resulted in a functional and accessible web application that addresses the core requirements of a clothing exchange platform.

The development of the platform involved practical implementation of several modern web development technologies and concepts. React.js was used to create the interactive frontend, while Node.js and Express.js were used to develop the backend services and RESTful APIs. MongoDB Atlas was integrated as the cloud database for storing and managing application data, and JWT-based authentication was implemented to secure user access and protected resources. Axios was used for communication between the frontend and backend, while Vercel and Render were used to deploy the frontend and backend respectively. The project also involved working with client-side routing, API integration, database operations, authentication, authorization, cloud deployment, and responsive user interface development.

Throughout the development and deployment process, the project involved solving several real-world technical challenges, including CORS configuration, JWT and cookie-based authentication, environment variable management, frontend-backend connectivity, MongoDB Atlas configuration, API errors, and routing issues after deployment. Resolving these challenges required systematic debugging, testing, and configuration of both development and production environments. The experience provided valuable practical understanding of how different components of a full-stack application interact and how deployment-related issues can affect an otherwise functional application. It also strengthened problem-solving skills and provided hands-on exposure to the development practices used in real-world software projects.